Quantum Fourier Transform and Quantum Phase estimation algorithms

Morten Hjorth-Jensen

FYS5419/9419: Week of March 23-27, 2026

Outline

1. From the classical DFT to the QFT, reminder from last week
2. Circuit decomposition with Hadamards and controlled phases
3. Approximate QFT and complexity
4. Quantum Phase Estimation (QPE)
5. Connection to Hamiltonian simulation
6. Connection to Variational Quantum Eigensolvers (VQE)
7. Themes for the second project

See also separate jupyter-notebooks at `https://github.com/CompPhysics/QuantumComputingMachineLearning/tree/gh-pages/doc/pub/week10/ipynb`

Motivation

The Quantum Fourier Transform (QFT) is one of the central algorithms in quantum computing.
It appears in:

- Shor's factoring algorithm,
- order finding,
- period finding,
- quantum phase estimation,
- several quantum simulation algorithms.

The QFT is the quantum analogue of the discrete Fourier transform, but it acts on the amplitudes of a quantum state.

Classical Discrete Fourier Transform, reminder from last week

Given a vector $x = (x_0, x_1, \ldots, x_{N-1})$, the discrete Fourier transform is

$$y_k = \frac{1}{\sqrt{N}} \sum_{j=0}^{N-1} x_j e^{2\pi ijk/N}, \qquad k = 0, \ldots, N-1.$$

Equivalently, the DFT matrix is

$$F_N = \frac{1}{\sqrt{N}} \begin{pmatrix} 1 & 1 & 1 & \cdots & 1 \\ 1 & \omega & \omega^2 & \cdots & \omega^{N-1} \\ 1 & \omega^2 & \omega^4 & \cdots & \omega^{2(N-1)} \\ \vdots & \vdots & \vdots & \ddots & \vdots \\ 1 & \omega^{N-1} & \omega^{2(N-1)} & \cdots & \omega^{(N-1)^2} \end{pmatrix}$$

with

$$\omega = e^{2\pi i/N}.$$

Definition of the QFT

For $N = 2^n$, the QFT is the unitary transformation

$$\mathrm{QFT}_N |x\rangle = \frac{1}{\sqrt{N}} \sum_{k=0}^{N-1} e^{2\pi i x k/N} |k\rangle , \qquad x = 0, \ldots, N-1.$$

Thus the QFT maps a computational basis state $|x\rangle$ to a coherent superposition of all basis states with phases determined by x. Because the map is linear, for a general state

$$|\psi\rangle = \sum_{x=0}^{N-1} c_x |x\rangle ,$$

we have

$$\mathrm{QFT}_N |\psi\rangle = \sum_{x=0}^{N-1} c_x \, \mathrm{QFT}_N |x\rangle .$$

Binary Representation of the Input

Let

$$x = x_1 x_2 \dots x_n$$

be the binary representation of the integer x, meaning

$$x = \sum_{j=1}^{n} x_j 2^{n-j}, \qquad x_j \in \{0, 1\}.$$

We also write binary fractions as

$$0.x_j x_{j+1} \dots x_n = \frac{x_j}{2} + \frac{x_{j+1}}{2^2} + \dots + \frac{x_n}{2^{n-j+1}}.$$

This notation is crucial because the QFT phases factor naturally into such binary fractions.

Step-by-Step Derivation of the Product Form I

Start from

$$\mathrm{QFT}_N |x\rangle = \frac{1}{\sqrt{2^n}} \sum_{k=0}^{2^n-1} e^{2\pi i x k/2^n} |k\rangle .$$

Write the output label k in binary:

$$k = k_1 k_2 \dots k_n, \qquad k = \sum_{m=1}^{n} k_m 2^{n-m}.$$

Then

$$\frac{xk}{2^n} = x \sum_{m=1}^{n} \frac{k_m}{2^m}.$$

Hence

$$e^{2\pi i x k/2^n} = \prod_{m=1}^{n} e^{2\pi i x k_m/2^m}.$$

Step-by-Step Derivation of the Product Form II

Because each $k_m \in \{0,1\}$, the sum over all k factorizes:

$$\mathrm{QFT}_N |x\rangle = \frac{1}{2^{n/2}} \bigotimes_{m=1}^{n} \left(|0\rangle + e^{2\pi i x/2^m} |1\rangle \right).$$

Now only the fractional part of $x/2^m$ matters in the phase, and that fractional part is exactly

$$0.x_{n-m+1}x_{n-m+2}\ldots x_n.$$

Thus

$$\mathrm{QFT}_N |x_1x_2\ldots x_n\rangle = \frac{1}{2^{n/2}} \left(|0\rangle + e^{2\pi i\, 0.x_n} |1\rangle \right) \otimes \left(|0\rangle + e^{2\pi i\, 0.x_{n-1}x_n} |1\rangle \right) \otimes \cdots$$

$$\cdots \otimes \left(|0\rangle + e^{2\pi i\, 0.x_1x_2\ldots x_n} |1\rangle \right).$$

The output order is therefore reversed relative to the input qubit order.

Final Product Form

The standard product form is

$$\mathrm{QFT}_N \left|x_1 x_2 \ldots x_n\right\rangle = \frac{1}{2^{n/2}} \left(\left|0\right\rangle + e^{2\pi i\, 0.x_n} \left|1\right\rangle\right) \otimes \left(\left|0\right\rangle + e^{2\pi i\, 0.x_{n-1}x_n} \left|1\right\rangle\right) \otimes \cdots$$

$$\cdots \otimes \left(\left|0\right\rangle + e^{2\pi i\, 0.x_1 x_2 \ldots x_n} \left|1\right\rangle\right).$$

This factorization is the key reason the QFT can be implemented efficiently.
It shows that the QFT can be built from:

- Hadamard gates,
- controlled phase rotations,
- a final qubit reversal using SWAP gates.

Basic Gates Used in the QFT

The main one-qubit gate is the Hadamard

$$H = \frac{1}{\sqrt{2}} \begin{pmatrix} 1 & 1 \\ 1 & -1 \end{pmatrix},$$

which creates equal superpositions.
The phase rotations are

$$R_k = \begin{pmatrix} 1 & 0 \\ 0 & e^{2\pi i/2^k} \end{pmatrix}.$$

Controlled-R_k gates apply R_k to a target qubit only if the control qubit is $|1\rangle$.

How the Circuit Arises

The first output qubit in the product form contains the phase

$$e^{2\pi i\, 0.x_1 x_2 \ldots x_n}.$$

This is produced by:

- a Hadamard on the first qubit, generating $\frac{1}{\sqrt{2}}(|0\rangle + |1\rangle)$,
- then a sequence of controlled phase gates from the later qubits, adding the binary-fraction phase.

The same pattern repeats recursively for the remaining qubits. Thus the QFT circuit is naturally built qubit by qubit.

Two-Qubit QFT: Mathematical Form

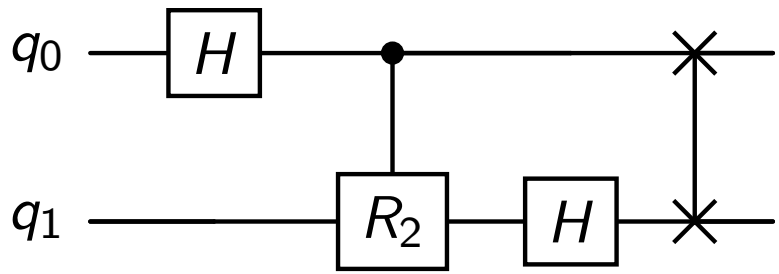
For $n = 2$, we have $N = 4$, and

$$\mathrm{QFT}_4 \left|x_1 x_2\right\rangle = \frac{1}{2}\left(\left|0\right\rangle + e^{2\pi i\, 0.x_2} \left|1\right\rangle\right) \otimes \left(\left|0\right\rangle + e^{2\pi i\, 0.x_1 x_2} \left|1\right\rangle\right).$$

This requires:

- Hadamard on the first qubit,
- controlled-R_2,
- Hadamard on the second qubit,
- then a SWAP to correct the reversed output order.

Two-Qubit QFT Circuit



Here:

- H creates the superposition,
- $R_2 = \mathrm{diag}(1, e^{i\pi/2})$,
- the SWAP reverses the qubit order.

Three-Qubit QFT: Mathematical Structure

For $n = 3$,

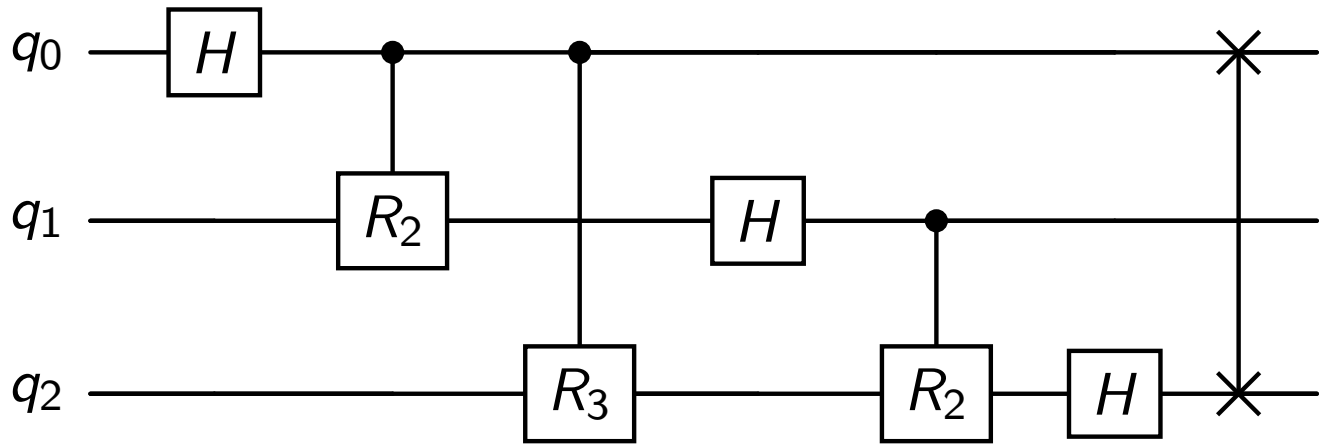
$$\mathrm{QFT}_8 \left|x_1 x_2 x_3\right\rangle = \frac{1}{\sqrt{8}} \left(\left|0\right\rangle + e^{2\pi i\, 0.x_3} \left|1\right\rangle\right) \otimes \left(\left|0\right\rangle + e^{2\pi i\, 0.x_2 x_3} \left|1\right\rangle\right) \otimes \left(\left|0\right\rangle + e^{2}\right.$$

The first qubit therefore requires phase contributions corresponding to

$$\frac{x_2}{2} + \frac{x_3}{4},$$

and this is exactly what the controlled R_2 and R_3 gates generate.

Three-Qubit QFT Circuit



Conceptually:

1. H on qubit 0, then controlled R_2, R_3
2. H on qubit 1, then controlled R_2
3. H on qubit 2
4. final reversal of qubit order

General n-Qubit QFT Circuit

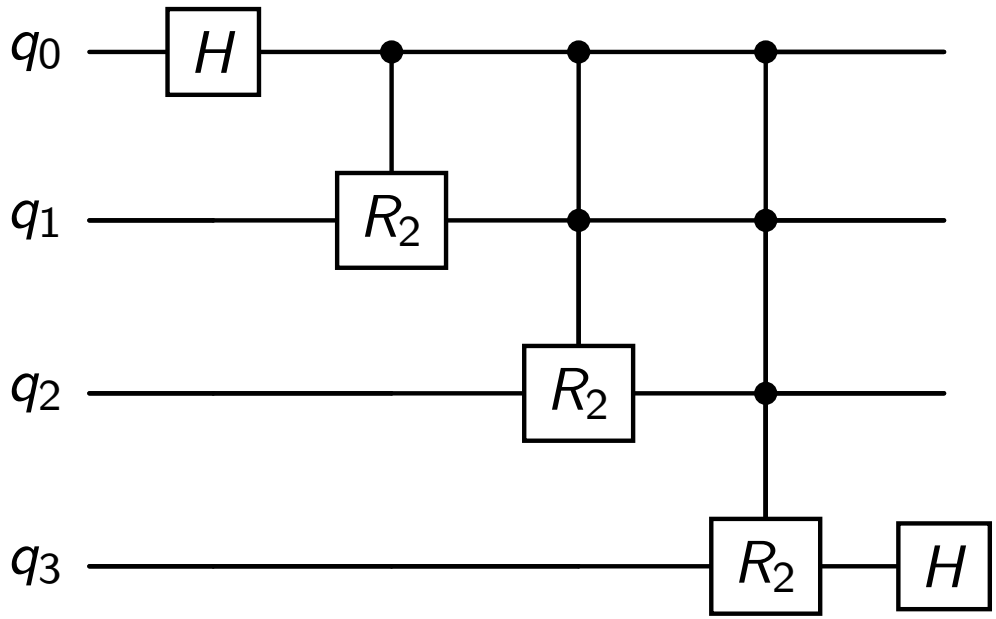
For qubit j, counting from the top:

1. Apply a Hadamard H
2. For each later qubit $k > j$, apply a controlled R_{k-j+1}
3. Continue recursively
4. At the end, reverse qubit order with SWAP gates

The general structure is:

$H \rightarrow$ controlled phases $\rightarrow H \rightarrow$ controlled phases $\rightarrow \cdots \rightarrow$ SWAP networ

General Pattern in Quantikz



This is a schematic pattern rather than a fully expanded full-n circuit. The key point is the nested structure of Hadamards and controlled phase gates.

Worked 3-Qubit Phase Example I

Take the computational basis input

$$|x\rangle = |101\rangle\,.$$

This means

$$x_1 = 1, \qquad x_2 = 0, \qquad x_3 = 1, \qquad x = 5.$$

The product form gives

$$\begin{aligned}\mathrm{QFT}_8\,|101\rangle =& \frac{1}{\sqrt{8}}\left(|0\rangle + e^{2\pi i\,0.1}\,|1\rangle\right) \otimes \left(|0\rangle + e^{2\pi i\,0.01}\,|1\rangle\right) \\ & \otimes \left(|0\rangle + e^{2\pi i\,0.101}\,|1\rangle\right).\end{aligned}$$

Worked 3-Qubit Phase Example II

Now evaluate the binary fractions:

$$0.1 = \frac{1}{2}, \qquad 0.01 = \frac{1}{4}, \qquad 0.101 = \frac{1}{2} + \frac{1}{8} = \frac{5}{8}.$$

Hence

$$e^{2\pi i\, 0.1} = e^{i\pi} = -1,$$

$$e^{2\pi i\, 0.01} = e^{i\pi/2} = i,$$

$$e^{2\pi i\, 0.101} = e^{2\pi i(5/8)} = e^{5\pi i/4}.$$

Therefore

$$\mathrm{QFT}_8 \left|101\right\rangle = \frac{1}{\sqrt{8}}(\left|0\right\rangle - \left|1\right\rangle) \otimes (\left|0\right\rangle + i \left|1\right\rangle) \otimes (\left|0\right\rangle + e^{5\pi i/4} \left|1\right\rangle).$$

Worked 3-Qubit Phase Example III

This example shows exactly how the QFT encodes the binary digits of the input into relative phases.
This phase encoding is what makes the QFT so useful in:

- phase estimation,
- period finding,
- extracting eigenphases of unitaries.

In other words, the QFT converts arithmetic structure into measurable interference structure.

Inverse QFT

The inverse QFT is obtained by reversing the circuit and taking Hermitian conjugates:

- replace each R_k by $R_k^\dagger$,
- reverse the order of the gates,
- keep the same SWAP network.

Thus the inverse QFT is also efficient.
In practice, many algorithms use the inverse QFT rather than the forward QFT.

Complexity of the QFT

For n qubits:

- n Hadamard gates,
- $\frac{n(n-1)}{2}$ controlled phase gates,
- about $\lfloor n/2 \rfloor$ SWAP gates.

Therefore the total gate count is

$$O(n^2).$$

This is exponentially more efficient than applying the classical DFT matrix directly to a 2^n-dimensional vector. The latter requires $O(N^2)$ operations, with $N = 2^n$. With three qubits we have already 9 versus 64 operations. With 100 qubits we have 10^4 compared with $2^{200} \sim 1.6 \times 10^{60}$.

Approximate QFT

Very small-angle phase gates contribute little.
Hence we may truncate the circuit by dropping controlled R_k gates for large k.
This gives the approximate QFT:

- fewer gates,
- shallower circuit,
- often still sufficient for practical algorithms.

With suitable truncation, the gate complexity can be reduced to approximately

$$O(n \log n).$$

Implementation

A typical implementation of the QFT proceeds as follows:

1. Choose the number of qubits n
2. Loop over qubits $j = 0, \ldots, n-1$
3. Apply H on qubit j
4. For each later qubit $k > j$, apply a controlled phase R_{k-j+1}
5. At the end, reverse the qubit order with SWAP gates

This is exactly the constructive logic derived from the product form. See the jupyter-notebook at `https://github.com/CompPhysics/QuantumComputingMachineLearning/tree/gh-pages/doc/pub/week10` for implementaions.

Pseudocode for QFT Construction

```
for j = 0 to n − 1 :
    apply H on qubit j
    for k = j + 1 to n − 1 :
        apply controlled-R_{k−j+1} from k to j
for j = 0 to ⌊n/2⌋ − 1 :
    swap qubit j with qubit n − 1 − j
```

Depending on software conventions, the control and target order may appear reversed relative to the mathematical derivation, but the logical structure is the same.

Simulation Remarks

When simulating the QFT numerically, it is useful to compare:

- the circuit output state,
- the exact matrix formula

$$\mathrm{QFT}_N |x\rangle = \frac{1}{\sqrt{N}} \sum_{k=0}^{N-1} e^{2\pi i x k/N} |k\rangle ,$$

- and the product-form prediction.

This provides a clean consistency check:

- amplitudes should have equal magnitude $1/\sqrt{N}$,
- phases should match the Fourier kernel,
- output qubit order must be checked carefully.

Common Source of Confusion: Bit Reversal

A frequent issue is the ordering of bits.
The product form naturally produces the qubits in reversed order:

$$(x_1 x_2 \dots x_n) \mapsto (\text{phase involving } x_n) \otimes \cdots \otimes (\text{phase involving } x_1 \dots x_n).$$

Thus:

- either include final SWAP gates,
- or accept the reversed output convention.

Many software libraries omit the final SWAPs and simply document the reversed ordering.

Why the QFT Matters Physically

The QFT is not just an abstract transform.
It provides a mechanism for:

- converting phase information into computational-basis information,
- exploiting interference to extract periodicity,
- connecting time evolution phases to measurable outputs.

In this sense, the QFT is a bridge between

$$\text{dynamical phase accumulation} \longrightarrow \text{algorithmic readout.}$$

Summary

The Quantum Fourier Transform:

- is the quantum analogue of the discrete Fourier transform,
- acts as
$$|x\rangle \mapsto \frac{1}{\sqrt{N}} \sum_k e^{2\pi i x k/N} |k\rangle ,$$
- factorizes into one-qubit phase states,
- is implemented using H, controlled R_k, and SWAP gates,
- has gate complexity $O(n^2)$,
- is central to phase estimation and many other algorithms.

Quantum Phase Estimation: Problem Statement

Suppose U is a unitary operator and $|\psi\rangle$ is one of its eigenstates:

$$U|\psi\rangle = e^{2\pi i\phi}|\psi\rangle, \qquad 0 \leq \phi < 1.$$

The goal of Quantum Phase Estimation (QPE) is to estimate the phase ϕ.
Equivalently, QPE extracts the eigenvalue of U in phase form.

Why QPE Matters

QPE is central because many physical problems can be mapped to eigenvalue estimation.
Examples:

- energy estimation in quantum chemistry,
- eigenvalue problems in many-body physics,
- Hamiltonian simulation,
- Shor's algorithm and period finding.

If we can encode a Hamiltonian into a unitary, then QPE can extract its eigenvalues.

Registers Used in QPE

QPE uses two registers:

- a control register of n qubits, initialized in $|0\rangle^{\otimes n}$,
- a system register, initialized in the eigenstate $|\psi\rangle$.

So the initial state is

$$|0\rangle^{\otimes n} |\psi\rangle .$$

The control register stores the estimated phase bits.

Step 1: Create a Uniform Superposition

Apply Hadamard gates to all control qubits:

$$|0\rangle^{\otimes n} \mapsto \frac{1}{2^{n/2}} \sum_{k=0}^{2^n-1} |k\rangle .$$

So the total state becomes

$$\frac{1}{2^{n/2}} \sum_{k=0}^{2^n-1} |k\rangle |\psi\rangle .$$

This prepares the control register as a coherent superposition of all binary numbers k.

Step 2: Controlled Powers of U

Now apply controlled powers of U:

$$U^{2^0}, U^{2^1}, U^{2^2}, \ldots, U^{2^{n-1}}.$$

Each control qubit determines whether the corresponding power is applied.
Because $|\psi\rangle$ is an eigenstate,

$$U^{2^j} |\psi\rangle = e^{2\pi i 2^j \phi} |\psi\rangle .$$

Hence each controlled unitary writes phase information into the control register.

Step 2: State After Controlled Powers

If the control register basis state is $|k\rangle$, then the controlled operations apply U^k, giving

$$U^k |\psi\rangle = e^{2\pi i k\phi} |\psi\rangle .$$

Therefore the combined state becomes

$$\frac{1}{2^{n/2}} \sum_{k=0}^{2^n-1} e^{2\pi i k\phi} |k\rangle |\psi\rangle .$$

The system register factors out, and the control register now contains the phase information.

Key Observation

The control register is in the state

$$\frac{1}{2^{n/2}} \sum_{k=0}^{2^n-1} e^{2\pi i k\phi} |k\rangle ,$$

which is exactly a Fourier-type state.
This is why the inverse QFT is the correct next step: it converts this phase-encoded superposition into a computational basis approximation of ϕ.

Step 3: Apply the Inverse QFT

Applying QFT^{-1} to the control register gives

$$\mathrm{QFT}^{-1}\left(\frac{1}{2^{n/2}}\sum_{k=0}^{2^n-1} e^{2\pi i k\phi}\,|k\rangle\right).$$

If

$$\phi = \frac{m}{2^n}$$

exactly for some integer m, then this becomes

$$|m\rangle\,.$$

So the control register stores the exact n-bit binary expansion of ϕ.

Exact QPE Case

Suppose

$$\phi = 0.\phi_1\phi_2\ldots\phi_n = \frac{m}{2^n}.$$

Then after inverse QFT, the control register becomes

$$|\phi_1\phi_2\ldots\phi_n\rangle\,.$$

Thus measuring the control register returns the exact binary digits of ϕ.
This is the cleanest case of QPE.

Step-by-Step Derivation with Binary Fractions

Let

$$\phi = 0.\phi_1\phi_2\ldots\phi_n\ldots$$

Then one can show that the control-register state after controlled powers factorizes as

$$\frac{1}{2^{n/2}}\left(|0\rangle + e^{2\pi i\, 2^{n-1}\phi}\,|1\rangle\right)\otimes\left(|0\rangle + e^{2\pi i\, 2^{n-2}\phi}\,|1\rangle\right)\otimes\cdots$$

$$\cdots\otimes\left(|0\rangle + e^{2\pi i\,\phi}\,|1\rangle\right).$$

Because

$$2^{n-1}\phi = \phi_1.\phi_2\phi_3\ldots,$$

only the fractional parts matter in the phases, so each qubit encodes one successive binary suffix.

How the Bits Emerge

Using the binary expansion of ϕ, the control-register state becomes

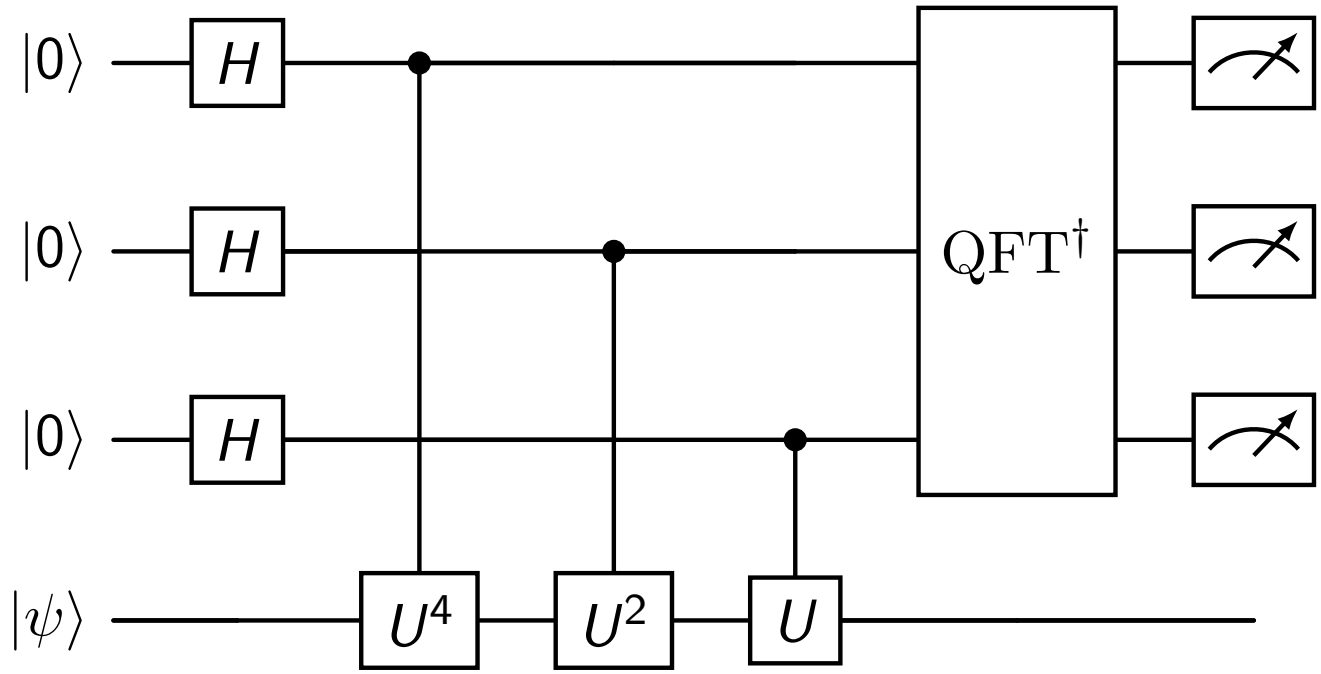
$$\frac{1}{2^{n/2}}\left(|0\rangle + e^{2\pi i\, 0.\phi_n}|1\rangle\right) \otimes \left(|0\rangle + e^{2\pi i\, 0.\phi_{n-1}\phi_n}|1\rangle\right) \otimes \cdots$$

$$\cdots \otimes \left(|0\rangle + e^{2\pi i\, 0.\phi_1\phi_2\ldots\phi_n}|1\rangle\right),$$

which has exactly the structure of a QFT state.
Therefore applying QFT^{-1} yields the bit string

$$\phi_1\phi_2\ldots\phi_n.$$

QPE Circuit



This example shows a 3-qubit control register and one system register.

Worked Example: $\phi = 5/8$

Suppose

$$\phi = \frac{5}{8} = 0.101.$$

Then the ideal QPE output on a 3-qubit control register is

$$|101\rangle .$$

Indeed,

$$e^{2\pi i \phi} = e^{2\pi i (5/8)}.$$

After the controlled powers and inverse QFT, the control register collapses to the bit string 101 with probability 1.

When ϕ Is Not Exactly Representable

If ϕ is not exactly of the form $m/2^n$, then QPE does not return one exact bit string.
Instead, the output is a probability distribution peaked near the best n-bit approximation:

$$\phi \approx \frac{m}{2^n}.$$

The measurement outcome gives the nearest binary approximation with high probability.

Precision of QPE

With n control qubits, QPE resolves phases to approximately

$$\Delta\phi \sim \frac{1}{2^n}.$$

Thus the precision improves exponentially with the number of control qubits.
This exponential precision is one of the main reasons QPE is so powerful.

From Phases to Energies

Suppose H is a Hamiltonian with eigenstate $|E\rangle$:

$$H|E\rangle = E|E\rangle.$$

Define the time-evolution operator

$$U(t) = e^{-iHt}.$$

Then

$$U(t)|E\rangle = e^{-iEt}|E\rangle.$$

Comparing with

$$U|\psi\rangle = e^{2\pi i\phi}|\psi\rangle,$$

we identify

$$\phi = -\frac{Et}{2\pi} \mod 1.$$

Thus QPE can estimate energies if we can implement e^{-iHt}.

Connection to Hamiltonian Simulation

QPE requires controlled applications of

$$U^{2^j} = e^{-iHt2^j}.$$

So QPE depends directly on the ability to perform Hamiltonian simulation.
In practice this may be done using:

- Trotter-Suzuki decompositions,
- linear-combination-of-unitaries methods,
- qubitization,
- problem-specific simulation techniques.

Thus:

$$\text{Hamiltonian simulation} + \text{QPE} \quad \Rightarrow \quad \text{energy estimation.}$$

Physical Interpretation

QPE measures the dynamical phase accumulated by an energy eigenstate under time evolution.
The algorithm turns the phase

$$e^{-iEt}$$

into a bit string encoding the energy.
So one can view QPE as a highly precise interferometric energy-measurement protocol.

Why QPE Is Important in Quantum Chemistry

In quantum chemistry and many-body physics, the main goal is often to compute eigenenergies of a Hamiltonian.
If we can prepare an approximate eigenstate with sufficient overlap with the true ground state, then QPE can project onto the exact eigenvalue with high precision.
This makes QPE a natural route to:

- ground-state energies,
- excited-state energies,
- spectral properties.

Connection to VQE

The Variational Quantum Eigensolver (VQE) and QPE aim at similar physics goals, but in different ways.

VQE:

- variational,
- hybrid quantum-classical,
- optimized by minimizing

$$E(\boldsymbol{\theta}) = \langle \psi(\boldsymbol{\theta}) | \, H \, | \psi(\boldsymbol{\theta}) \rangle \, ,$$

- suitable for noisy near-term devices.

QPE:

- not variational,
- requires coherent controlled time evolution,
- provides direct eigenvalue estimation,
- suited to fault-tolerant quantum computers.

How VQE and QPE Complement Each Other

A common long-term strategy is:

1. Use a variational method such as VQE to prepare a good approximate eigenstate

   $$|\psi_{\text{trial}}\rangle$$

   with large overlap with the true eigenstate.
2. Use that state as the input to QPE.
3. QPE then refines the energy estimate to high precision.

So VQE can be viewed as a state-preparation front end for QPE.

Overlap Requirement

If the initial state is

$$|\psi_{\text{trial}}\rangle = \sum_j c_j |E_j\rangle ,$$

then QPE outputs E_j with probability

$$|c_j|^2.$$

Therefore the success probability depends on the overlap with the desired eigenstate.
This is exactly why improved ansätze in VQE are also valuable for QPE-based workflows.

QPE vs VQE in Practice

VQE

- shallow circuits,
- many repeated measurements,
- classical optimization loop,
- noise tolerant,
- limited by optimization difficulty and shot cost.

QPE

- deep coherent circuits,
- requires controlled e^{-iHt},
- very precise,
- ideal for fault-tolerant hardware,
- often viewed as the long-term gold standard for energy estimation.

Semiclassical Interpretation

One may interpret QPE as repeatedly extracting successive binary digits of a phase.
The inverse QFT acts globally, but conceptually it decodes the phase one bit at a time.
This bitwise structure is one reason QPE is so elegant mathematically.

Implementation

A implementation of the QPE typically proceeds as follows:

1. choose the number of control qubits,
2. initialize the system register in an eigenstate or approximate eigenstate,
3. apply Hadamards to the control register,
4. apply controlled U^{2^j},
5. apply inverse QFT,
6. measure the control register,
7. interpret the measurement as an estimate of ϕ.

If the notebook also implements the QFT explicitly, it will usually reuse the same Hadamard + controlled-phase + swap pattern.

Summary of the Full Picture

$$\text{Hamiltonian } H \quad \longrightarrow \quad U(t) = e^{-iHt} \quad \longrightarrow \quad \text{QPE} \quad \longrightarrow \quad E$$

The QFT is the phase-decoding engine inside QPE.
VQE and QPE are complementary:

- VQE is a near-term variational energy estimator,
- QPE is a long-term high-precision eigenvalue estimator,
- VQE can provide input states for QPE.

Summary

In these notes we have seen that:

- the QFT maps

$$|x\rangle \mapsto \frac{1}{\sqrt{N}} \sum_k e^{2\pi i x k/N} |k\rangle ,$$

- QPE uses controlled powers of a unitary and the inverse QFT to extract eigenphases,
- for Hamiltonian evolution

$$U(t) = e^{-iHt},$$

  QPE becomes an energy-estimation algorithm,
- VQE and QPE address related eigenvalue problems with different hardware requirements.

Suggestions for the second project

- Continuation of project 1 with more realistic systems and using adaptive VQE on actual (real) quantum computers
- Implementation and studies of the Quantum Approximate Optimization Algorithm (QAOA): Simulation of linear algebra systems on quantum computers, solving for example differential equations
- Implementation of the HHL algorithm (needs QPE)
- Applications and implementations of quantum machine learning algorithms
- Implement quantum neural networks for PINNs
- Detailed quantum chemistry applications.

Suggestions for the second project

- Studies of entanglement and physical realization of quantum gates and circuits
- Implementing Shor's algorithm or other algorithms
- Parallelizing the Variational Quantum Eigensolver with multi-GPU scaling, see `https://arxiv.org/abs/2601.09951`
- Implementing and comparing the quantum phase estimation algorithm with VQE, see for example recent QPE paper at `"https://arxiv.org/abs/2601.05788`
- Studies of hyperparameters in VQE, see for example `https://arxiv.org/abs/2601.16679`
- Implementing unitary coupled cluster theory on a quantum computer, see `https://arxiv.org/abs/2501.15893`

What is the QAOA algorithm?

QAOA stands for the Quantum Approximate Optimization Algorithm

Each part of the name reflects something essential about the method:

- Quantum: it runs on a quantum computer, using qubits and quantum gates.
- Approximate: it typically finds near-optimal solutions rather than exact ones.
- Optimization: it is designed to solve optimization problems (like MaxCut, scheduling, portfolio selection).
- Algorithm: it is a well-defined hybrid quantum–classical procedure.

QAOA is a variational quantum algorithm that uses a parameterized quantum circuit to search for good solutions to hard optimization problems.

What is the HHL algorithm?

The HHL algorithm—named after Aram Harrow, Avinatan Hassidim, and Seth Lloyd—is one of the central quantum algorithms for solving linear systems of equations.
The core idea is to solve a linear system

$$A\mathbf{x} = \mathbf{b}.$$